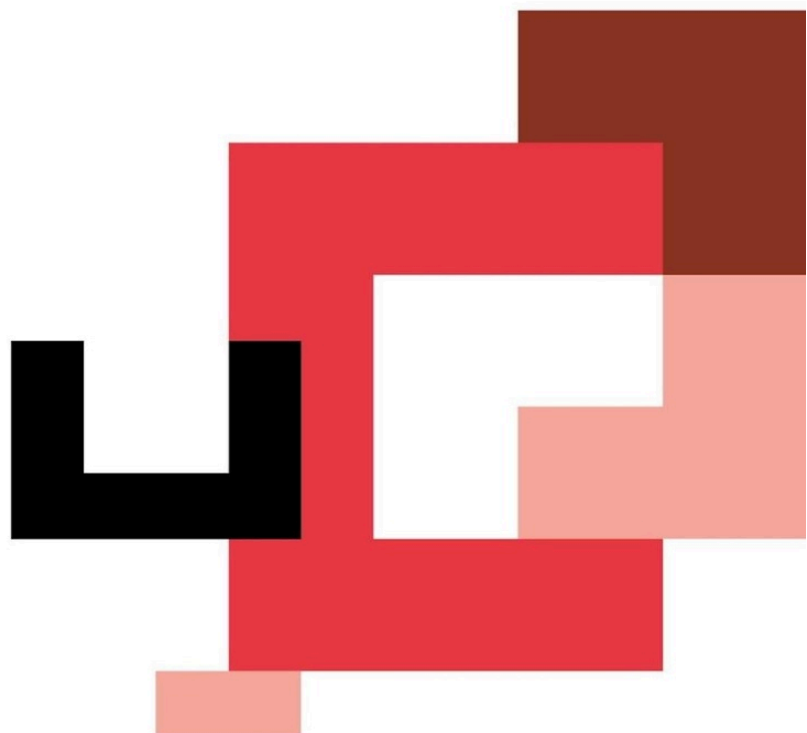




FULL STACK DEVELOPER + IA

Módulo 3 | Feature 4

useMemo



Objetivo de la feature

Optimizar la aplicación y cerrar el flujo del cliente, entendiendo:

- Cómo usar `useMemo` para evitar cálculos innecesarios
 - Cómo trabajar con datos derivados (filtros, ordenación)
 - Cuándo optimizar y cuándo no
 - Cómo construir una experiencia completa de compra
-

Contexto del sprint

En el Sprint 3:

- Ya tenemos estado global (Redux)
- Autenticación funcionando
- Carrito y wishlist completos

En este sprint:

- Optimizamos rendimiento
- Mejoramos experiencia de usuario
- Cerramos el flujo de compra

Este sprint actúa como:

- cierre del bloque de frontend
 - consolidación de todo lo aprendido
-

Requisitos

Tech / configuración

- Backend en `http://localhost:3000`
 - React + Redux Toolkit
 - Axios con interceptors funcionando
-

Qué hay que hacer (paso a paso)

1) Optimizar filtros con useMemo

En páginas como `ProductsPage`:

- Filtrar productos
- Ordenar productos

Problema sin `useMemo`:

- Se recalcula en cada render

Solución:

```
JavaScript
const filteredProducts = useMemo(() => {

  return products

    .filter(...)

    .slice()

    .sort(...)

}, [products, search, sortBy])
```

Objetivo: recalcular solo cuando cambian dependencias.

2) Entender datos derivados

- `products` → estado base
- `filteredProducts` → estado derivado

Regla:

- No guardar derivados en estado
- Calcularlos con `useMemo`

Objetivo: evitar inconsistencias.

3) Cerrar flujo del cliente

Asegurar que funciona:

- Navegación completa
- Carrito
- Wishlist
- Checkout
- Feedback visual

Objetivo: app usable de principio a fin.

4) Mejorar experiencia de usuario

- Estados de loading
- Manejo de errores
- Feedback visual
- Navegación clara

Objetivo: hacer la aplicación usable y consistente.

Uso de la IA en este sprint

Para guiarte

- Cuándo usar useMemo
 - Revisión de lógica de filtros
 - Debug de estados y render
-

Prompts útiles

useMemo

Estoy usando useMemo para filtrar productos.
¿Cómo sé si realmente lo necesito o es innecesario?

Debug

Tengo problemas con renders innecesarios en React.
Te paso el código. Dime posibles causas.

Pistas

useMemo

Usarlo cuando:

- Hay cálculos sobre arrays
- El componente re-renderiza mucho

No usarlo para:

- operaciones simples
-

sort()

Siempre copiar antes:

```
JavaScript  
.slice().sort()
```

Datos derivados

- No se guardan en estado
 - Se calculan
-

Importante que tienen que tener en cuenta

No optimizar antes de tiempo

useMemo no siempre es necesario.

No mutar estado

Nunca modificar arrays directamente.

Redux vs estado local

- Global → Redux
- Local → useState

Flujo completo

La app debe funcionar de principio a fin:

usuario → carrito → checkout → confirmación

Checks rápidos (autoevaluación)

- Filtros funcionan correctamente
 - No hay renders innecesarios visibles
 - Carrito completo
 - Checkout funciona
 - La navegación es fluida
 - La UI responde correctamente a los estados
-

Enfoque del sprint

- Entender optimización
- Consolidar conocimientos
- Pensar en arquitectura

Este sprint cierra el bloque técnico principal antes de pasar a una fase más autónoma del proyecto.